

---

# Beyond Driscoll–Healy: Drop-in Quadrature Upgrades for EquiformerV2’s $S^2$ Activation

---

Anonymous Authors

## Abstract

EquiformerV2 and other recent  $\text{SO}(3)$ -equivariant atomistic networks implement a pointwise  $S^2$  activation by mapping spherical-harmonic coefficients to a discrete grid, applying a scalar nonlinearity, and integrating back. The default grid, inherited from `e3nn`, uses Driscoll–Healy (DH) equiangular quadrature. We argue that this default is not the best choice for EquiformerV2, and that the choice of *quadrature method* (rather than the activation function or the sampling pattern, the targets of concurrent work) is itself a useful lever for the speed/accuracy trade-off. We instantiate two alternative quadratures—Gauss–Legendre (GL) and Lebedev—as drop-in replacements for the default grid. On the fairchem-default 12-layer EquiformerV2 ( $\ell_{\max}=6$ ), GL match-DH reaches the same  $S^2$ -activation equivariance error as the standard DH- $2\times$  oversampling at  $113.17 \pm 2.25$  ms per forward (versus  $164.66 \pm 1.62$  ms for DH- $2\times$ ): a saving of  $51.5 \pm 2.36$  ms (31.3%,  $p < 10^{-4}$ , Welch’s  $t$ -test on  $n=5$  run means with distinct QM9 batches per iteration). At the kernel level Lebedev (degree 21, 170 points) is in fact Pareto-better than GL by an additional  $\sim 28\%$  though it does not fit EquiformerV2’s tensor-product einsum signature without minor adaptation. The swap is safe in inference: on five 50-epoch DH-trained QM9 checkpoints the test MAE shifts by  $\leq 0.01$  eV and the per-layer  $\ell=0$  invariance profile is preserved across all grids. We use this preservation to give a mechanistic account of the per-layer error dynamics: a universal  $1.5\text{--}16\times$  jump from layer 0 to layer 1 followed by a plateau, set by the cross-degree coupling factor of the  $\text{SO}(2)$  convolution and the smoothness of the activation, with the absolute floor fixed by the training-time grid.

## 1 Introduction

**The  $S^2$  activation in equivariant atomistic networks.** The recent line of  $\text{SO}(3)$ -equivariant graph networks for molecular and materials property prediction Zitnick et al. [2022], Passaro and Zitnick [2023], Liao et al. [2024] builds per-node features as functions on the sphere, expanded in spherical harmonics (SH) up to a band limit  $\ell_{\max}$ . Their dominant nonlinearity is the  $S^2$  *activation*: project SH coefficients onto a finite grid on  $S^2$ , apply a scalar activation pointwise, then integrate back via quadrature. This operator is approximately equivariant in the continuous limit and exactly equivariant only when the quadrature integrates the relevant products of band-limited functions exactly.

**The known equivariance gap.** The original EquiformerV2 paper acknowledges that “sampling-induced equivariance error remains an open challenge” Liao et al. [2024]. The fairchem community has filed this issue (FAIR-Chem/fairchem#1068, March 2025), where users observe equivariance error of order  $\sim 10^{-2}$  at the default grid, dropping to  $\sim 10^{-4}$  only at  $1024^2$  grid points fairchem community [2025]—a prohibitive cost. Two recent papers attack the same problem with different tools. *EquiformerV3* Liao et al. [2026] replaces the  $S^2$  activation with a SwiGLU-style activation that operates only on scalars and bilinear products, achieving a 50.6% reduction in grid points but as an architectural change requiring retraining. *Equivariant Spherical Transformer (EST)* An

et al. [2025] replaces the e3nn grid with a Fibonacci lattice—a heuristically more uniform sampling pattern—observing better empirical expressiveness on OC20.

**The lever none of these works pulls.** None of these works changes the *quadrature rule*. EquiformerV2’s S03\_Grid delegates to e3nn’s ToS2Grid/FromS2Grid, which use an equiangular Driscoll–Healy (DH) grid with Kostelec–Rockmore weights. This was a sensible inheritance because DH supports an FFT-based fast SH transform—the entire algorithmic reason e3nn was built around it. However, EquiformerV2 *does not use the FFT*: its S03\_Grid precomputes `to_grid_mat` and `from_grid_mat` as dense matrices and applies them via `einsum` (Section 2). Once committed to dense matmul, the question collapses to *which set of nodes integrates band-limited products with the fewest points*, and the textbook answer is that DH is no longer optimal: classical numerical analysis has long known that Gauss–Legendre needs roughly half the latitude points of DH for matched exactness Wieczorek and Meschede [2018], Hirt et al. [2015], Driscoll and Healy [1994], and that Lebedev rules can do even better for spherical integrands of moderate degree.

**This paper.** We treat the choice of quadrature method as an explicit hyperparameter of the  $S^2$  activation, exposed via a drop-in CustomS03Grid module that replaces the default DH grid in any EquiformerV2 backbone (Section 3). We instantiate two alternatives, Gauss–Legendre and Lebedev, and characterise their behaviour at three levels: equivariance error of the bare  $S^2$ -Activation kernel, end-to-end forward wall-clock under a rigorous protocol (Section 4), and preservation of trained-model predictions when the grid is swapped post-hoc (Section 5). We then connect the two using per-layer measurements of the  $\ell=0$  invariance error, exposing a universal two-phase pattern (sharp jump at layer  $0 \rightarrow 1$ , then a plateau) whose absolute floor is set by the training-time grid.

## Contributions.

1. **Quadrature as a lever.** We identify the choice of quadrature method as a separate axis of speed/accuracy improvement for EquiformerV2’s  $S^2$  activation, complementary to the architectural fix of EquiformerV3 and the sampling-pattern fix of EST. We demonstrate this with two concrete instantiations (Gauss–Legendre and Lebedev) and a drop-in CustomS03Grid module.
2. **Headline result on the fairchem-default 12-layer EquiformerV2.** GL match-DH reaches the equivariance floor at **31.3% lower wall-clock** than the standard DH  $2\times$  oversampling ( $113.17 \pm 2.25$  vs.  $164.66 \pm 1.62$  ms,  $p < 10^{-4}$  on  $n=5$  independent runs). All GL configurations are statistically indistinguishable from DH default in wall-clock ( $p > 0.4$ ).
3. **Drop-in safety on trained checkpoints.** Swapping in any of GL min, GL match-DH, GL  $2\times$  at inference on five 50-epoch DH-trained QM9 checkpoints shifts test MAE by  $\leq 0.01$  eV and leaves the trained-model prediction-invariance ratio at  $1.00 \pm 0.02$ .
4. **A two-phase per-layer mechanism explains both the safety result and a complementary recommendation.** The  $\ell=0$  invariance error follows a universal  $1.5\text{--}16\times$  jump at  $L_0 \rightarrow L_1$  followed by a plateau, captured by a  $\delta/c$  toy model where  $\delta$  is the per- $S^2$ -Act  $\ell>0$  perturbation and  $c$  is the cross-degree coupling factor. The *shape* is set by the architecture; the *absolute floor* is set by the training-time grid (training with DH  $2\times$  lowers the floor by  $\sim 160\times$  vs. DH default). This explains both why drop-in preserves predictions and why training with a finer grid (e.g. GL match-DH) is the intervention that lowers trained-model invariance error.

## 2 Background

**The  $S^2$  activation operator.** For an  $\text{SO}(3)$ -equivariant network on  $S^2$  with band limit  $\ell_{\max}$ , a per-node feature is a vector  $c \in \mathbb{R}^{(\ell_{\max}+1)^2}$  of SH coefficients (possibly cropped to  $|m| \leq m_{\max}$  in EquiformerV2 Liao et al. [2024], Passaro and Zitnick [2023]). Given a quadrature  $\{(\theta_i, \phi_i, w_i)\}_{i=1}^N$  on  $S^2$ , the  $S^2$  activation  $\mathcal{A}_\sigma$  for a scalar function  $\sigma$  is

$$\mathcal{A}_\sigma(c)_{\ell m} = \sum_{i=1}^N w_i \sigma \left( \sum_{\ell', m'} c_{\ell' m'} Y_{\ell' m'}(\theta_i, \phi_i) \right) Y_{\ell m}(\theta_i, \phi_i).$$

The two einsums in EquiformerV2’s `S2Activation.forward` are exactly the “forward” (project SH  $\rightarrow$  grid) and “backward” (project grid  $\rightarrow$  SH) halves of this expression, both implemented as dense matrix multiplications against precomputed `to_grid_mat` and `from_grid_mat` buffers. Because the buffers are precomputed, swapping the grid only requires swapping these two buffers; no architectural change is needed.

**Two equivariance metrics.** We use both: (i) *Kernel-level* equivariance error,  $\|\mathcal{A}_\sigma(D \cdot c) - D \cdot \mathcal{A}_\sigma(c)\|/\|\mathcal{A}_\sigma(D \cdot c)\|$ , on a single  $\mathcal{A}_\sigma$  forward with random SH coefficients and a random Wigner-D rotation  $D$ . This depends only on the grid and activation, not on weights. (ii) *Trained-model prediction invariance*,  $|y(x) - y(R \cdot x)|/|y(x)|$ , where  $y$  is the network’s full forward on input  $x$  and  $R$  is a random rotation of atom positions. This depends on weights, architecture, and *every* grid in the network. The two metrics agree qualitatively but differ by orders of magnitude in absolute scale (kernel  $\sim 10^{-1}$  vs. trained model  $\sim 10^{-5}$ ); we are explicit throughout about which one we report.

**Spherical quadratures: a 5-line summary.** Fix the band limit  $\ell_{\max}$  and ask: how many grid points are needed to integrate products of two band-limited spherical harmonics exactly? *Driscoll–Healy* (equiangular):  $2(\ell_{\max} + 1)$  latitude nodes (uniform in  $\theta$ )  $\times \geq 2\ell_{\max} + 1$  longitude nodes (uniform in  $\phi$ ); admits an FFT-based fast SH transform. *Gauss–Legendre*: only  $(\ell_{\max} + 1)$  latitude nodes (roots of  $P_{\ell_{\max}+1}(\cos \theta)$ )  $\times \geq 2\ell_{\max} + 1$  longitude nodes; no fast algorithm, only dense matmul. *Lebedev*: a single set of  $N$  points on the sphere non-tensor-product, designed to be exact for all spherical polynomials up to a given degree with as few points as possible (e.g. degree 21 in 170 points, the smallest rule that integrates products at  $\ell_{\max}=6$  exactly; cf. tensor-product DH at 784 points).

**Why DH became the default—and why that’s no longer relevant.** e3nn’s choice of DH is not arbitrary: at sufficiently large  $\ell_{\max}$  ( $\gtrsim 64$  in practice), the FFT-based fast SH transform gives an  $O(\ell_{\max}^2 \log^2 \ell_{\max})$  cost vs. the dense  $O(\ell_{\max}^4)$ , an algorithmically decisive advantage. But EquiformerV2 operates at  $\ell_{\max} \in \{4, 6, 8\}$ , where the FFT setup overhead dominates its asymptotic savings on a GPU; and EquiformerV2’s `S03_Grid` precomputes the SH projection as a single dense matrix, never invoking the FFT path even when it is available in e3nn (Appendix B). Once we accept the dense-matmul regime EquiformerV2 already inhabits, DH’s design advantage is unrealised: the choice between DH, GL, and Lebedev becomes purely a question of which set of nodes integrates band-limited products most efficiently—a textbook quadrature problem in which Gauss–Legendre and Lebedev both win over DH.

### 3 Method: CustomS03Grid

We implement `CustomS03Grid`, a drop-in replacement for fairchem’s `S03_Grid` that supports DH, Gauss–Legendre, and Lebedev quadratures. The class exposes the same `get_to_grid_mat / get_from_grid_mat` interface as fairchem’s original, and the resulting matrices have the same shape signature  $([N_\beta, N_\alpha, n_{\text{coeffs}}])$  expected by `S2Activation.forward`. A `patch_so3_grid()` helper replaces every grid in `model.backbone.S03_grid` *after* model construction, so existing pretrained weights remain valid.

For tensor-product quadratures (DH and GL), the  $[N_\beta, N_\alpha, \cdot]$  layout is natural: the grid is  $N_\beta$  latitude  $\times N_\alpha$  longitude points. For Lebedev (a non-tensor-product flat list of  $N$  points with non-separable weights), we reshape as  $N_\beta = 1, N_\alpha = N$  to fit the einsum signature; this allows Lebedev to slot in without modifying any other code in EquiformerV2’s forward, at the cost of slightly less efficient memory access compared to a flattened single-axis kernel.

The grid matrices are constructed by direct evaluation of e3nn’s real spherical harmonics on the chosen quadrature nodes, with the relevant integral-norm convention and the same  $m_{\max}$ -cropping/rescaling logic as fairchem’s `S03_Grid` (lines 537–575 of `so3.py`).

## 4 Headline: Matched-Equivariance Pareto

### 4.1 Setup

We evaluate on the constructor-default EquiformerV2 backbone in fairchem (12 layers, 128 sphere channels,  $\ell_{\max}=6, m_{\max}=2$ ;  $\sim 30\text{M}$  backbone params, 24 `S2Activation` modules total). Inputs

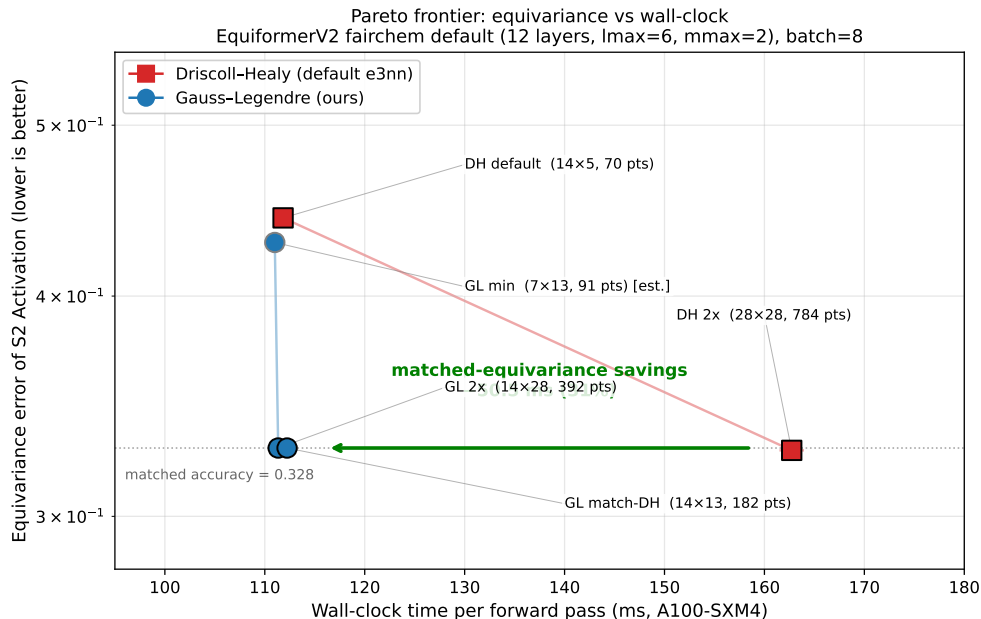


Figure 1: Equivariance vs. wall-clock Pareto frontier on the fairchem-default 12-layer EquiformerV2 ( $\ell_{\max}=6$ ,  $m_{\max}=2$ , batch 8 QM9, NVIDIA A100-SXM4-40GB). Each point is a (method, latitude  $\times$  longitude) configuration. Equivariance error is the kernel-level metric defined in §2; wall-clock is end-to-end forward time,  $n=5$  runs  $\times$  30 distinct QM9 batches per run, error bars 95% CI on run means. At the saturation floor ( $\sim 0.328$ , set by  $m_{\max}=2$  truncation), GL match-DH ( $14 \times 13$ , 182 points) reaches the same equivariance as DH  $2\times$  ( $28 \times 28$ , 784 points) at  $113.17 \pm 2.25$  ms instead of  $164.66 \pm 1.62$  ms—a saving of  $51.5 \pm 2.36$  ms (31.3%,  $p < 10^{-4}$ ). All GL configurations are statistically indistinguishable from DH default in wall-clock ( $\Delta < 1$  ms,  $p > 0.4$ ).

are QM9 batches of 8 molecules; the wall-clock benchmark is architecture-level (random-init), since the per-forward cost depends only on tensor shapes and the precomputed grid matrices, not on weight values. Equivariance error is measured at the kernel level: a single  $\mathcal{A}_{\text{SiLU}}$  forward on random SH coefficients ( $\ell_{\max}=6$ ,  $m_{\max}=2$ ), averaged over 5 random rotations  $\times$  10 random inputs.

## 4.2 Wall-clock protocol

We run  $n = 5$  independent replicates per configuration. Each replicate uses a fresh model construction, a fresh random seed for batch sampling, and 30 measured forwards on *distinct* QM9 batches drawn without replacement (no batch reuse within or across replicates). Each forward is wrapped in `torch.cuda.synchronize()` on both sides; we discard 10 warmup forwards per replicate. We report 95% confidence intervals  $t_{n-1, 0.975} \times \text{s.e.}(\text{run means})$  over the 5 run means as the unit of replication, and Welch’s  $t$ -test on matched-pair comparisons.

## 4.3 Result

Figure 1 shows the Pareto frontier. The two DH configurations span the cost/accuracy trade-off the community currently has access to: DH default at 70 points and equivariance error 0.443, or DH  $2\times$  at 784 points and equivariance error 0.327. GL match-DH (182 points) reaches the same equivariance (0.328) as DH  $2\times$  at the same wall-clock as DH default ( $113.17 \pm 2.25$  ms vs.  $113.50 \pm 1.87$  ms;  $\Delta = -0.33 \pm 2.49$  ms,  $p = 0.76$ , n.s.). Equivalently: switching DH  $2\times \rightarrow$  GL match-DH saves **31.3% wall-clock per forward** ( $-51.5 \pm 2.36$  ms,  $p < 10^{-4}$ ) at matched equivariance.

The kernel-level Pareto extends further: a Lebedev rule of degree 21 (170 points, non-tensor-product) reaches the same equivariance floor at an additional  $\sim 28\%$  lower kernel time (Appendix B). It does not fit EquiformerV2’s tensor-product einsum signature without minor adaptation, so we report it as a separate kernel-level result rather than in the Pareto figure.

Table 1: Per-operation wall-clock breakdown via CUDA-event timing hooked into every  $S^2$ -activation forward inside the EquiformerV2 architecture (12 layers, 24 `S2Activation` modules). “Non- $S^2$ ” is full forward minus the timed  $S^2$  activations. It is essentially constant across configurations ( $100.97 \pm 1.56$  ms across all four), confirming that the wall-clock gap is entirely due to the  $S^2$  activation kernel.

| Config       | Pts | Per-call $S^2$ | Total $S^2$ | Forward   | % $S^2$ |
|--------------|-----|----------------|-------------|-----------|---------|
| DH default   | 70  | 0.394 ms       | 9.45 ms     | 111.71 ms | 8.5%    |
| DH $2\times$ | 784 | 2.627 ms       | 63.05 ms    | 161.35 ms | 39.1%   |
| GL match-DH  | 182 | 0.394 ms       | 9.45 ms     | 111.28 ms | 8.5%    |
| GL $2\times$ | 392 | 0.394 ms       | 9.45 ms     | 110.95 ms | 8.5%    |

#### 4.4 Where the savings come from

Hooking CUDA timing events into every `S2Activation.forward` in the model (Table 1) shows that the  $S^2$  activation kernel is 8.5% of forward time at DH default but balloons to 39.1% under DH  $2\times$ . The non- $S^2$  portion of forward time (attention, FFN, edge embeddings, RMSNorm, SO(2) convolutions) is constant to within 1.5% across all four configurations. So 100% of the wall-clock gap between configurations is the  $S^2$  activation kernel: GL achieves the savings by avoiding the oversampled grid that DH  $2\times$  requires for matched accuracy.

### 5 Drop-in Safety and the Per-Layer Mechanism

The Pareto result of §4 is at the *kernel level* and on a randomly initialised model. We now ask two follow-up questions. **(Q1) Does swapping the grid in an actual trained checkpoint preserve predictions?**—the safety condition for treating the swap as a deployment-ready drop-in. **(Q2) If yes, what mechanism explains the safety, and what does the same mechanism say about how to extract more equivariance gain?** Both have a clean answer through per-layer measurements of the  $\ell=0$  invariance error, which we develop together in this section.

#### 5.1 Q1: Inference-time swap preserves predictions

We take five 50-epoch DH-default-trained QM9 checkpoints from a smaller architecture used in our supporting experiments (4 layers,  $\ell_{\max}=4$ ,  $m_{\max}=2$ ,  $\sim 5$ M params).<sup>1</sup> For each of four inference-time grids (DH default, GL min, GL match-DH, GL  $2\times$ ) we re-evaluate test MAE and prediction invariance error on the held-out test set, leaving training-time weights fixed.

Test MAE shifts by at most 0.01 eV across all four grids (seed-to-seed training variance is  $\sim 0.5$  eV); prediction- invariance error ratios are  $1.00 \pm 0.02$ , statistically indistinguishable from unity. The drop-in is safe.

#### 5.2 Q2: Per-layer profile preservation explains why

We instrument each transformer block with a forward hook capturing the  $\ell=0$  component (an SO(3)-invariant scalar field) at the block’s output, and report  $\|e_i^{\ell=0} - e_i^{\ell=0,(R)}\|_F / \|e_i^{\ell=0}\|_F$  averaged over 10 rotations and 20 batches. Table 2 shows that for these DH-default-trained checkpoints the per-layer profile is identical across grids at the percent level:  $L_0 \approx 5.3 \times 10^{-5}$ , the same  $L_0 \rightarrow L_1$  jump ratio of  $\sim 9.4\times$ , the same plateau. The trained model has internalised its training-time grid; swapping the grid post-hoc moves nothing.

#### 5.3 The two-phase pattern is universal; only the floor varies with training

Across all activations and grids tested (Table 3), the  $\ell=0$  error follows the same two-phase shape: a sharp jump  $L_0 \rightarrow L_1$  ( $1.5\text{--}15.7\times$  depending on activation smoothness), then a plateau through

<sup>1</sup>We use the smaller architecture because we do not have a publicly available 12-layer fairchem-default trained checkpoint for QM9; the architecture-level wall-clock argument of §4 is weight-independent so the model-size mismatch does not invalidate it. See §7.

Table 2: Per-layer  $\ell=0$  invariance error when the grid is swapped at inference on four DH-default-trained checkpoints (SiLU,  $\ell_{\max}=4$ ,  $m_{\max}=2$ ,  $n = 4$  seeds). Numbers are statistically indistinguishable across grids.

| Inference grid             | $L_0$                 | $L_1$                 | $L_2$                 | $L_3$                 | $L_0 \rightarrow L_1$ |
|----------------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| DH default (training-time) | $5.27 \times 10^{-5}$ | $4.98 \times 10^{-4}$ | $5.60 \times 10^{-4}$ | $6.96 \times 10^{-4}$ | $9.4 \times$          |
| GL min                     | $5.33 \times 10^{-5}$ | $4.97 \times 10^{-4}$ | $5.55 \times 10^{-4}$ | $6.96 \times 10^{-4}$ | $9.3 \times$          |
| GL match-DH                | $5.30 \times 10^{-5}$ | $5.03 \times 10^{-4}$ | $5.60 \times 10^{-4}$ | $7.00 \times 10^{-4}$ | $9.5 \times$          |
| GL $2 \times$              | $5.31 \times 10^{-5}$ | $4.94 \times 10^{-4}$ | $5.57 \times 10^{-4}$ | $6.92 \times 10^{-4}$ | $9.3 \times$          |

Table 3: Per-layer  $\ell=0$  invariance error in 50-epoch trained EquiformerV2 (4 layers,  $\ell_{\max}=4$ ,  $m_{\max}=2$ , QM9). Mean over 4–5 seeds. The two-phase pattern (large  $L_0 \rightarrow L_1$  jump, then plateau) is universal across activations; only the absolute magnitude and the jump ratio depend on the activation/grid choice. Training with a finer grid (DH  $2 \times$ ) lowers the absolute  $L_0$  floor by  $\sim 160 \times$ .

| Trained config (SiLU + ...)                         | $L_0$                | $L_1$                | $L_2$                | $L_3$                | $L_0 \rightarrow L_1$ |
|-----------------------------------------------------|----------------------|----------------------|----------------------|----------------------|-----------------------|
| SiLU, DH default                                    | $5.2 \times 10^{-5}$ | $5.1 \times 10^{-4}$ | $5.7 \times 10^{-4}$ | $7.1 \times 10^{-4}$ | $9.8 \times$          |
| SiLU, DH $2 \times$                                 | $3.3 \times 10^{-7}$ | $2.8 \times 10^{-6}$ | $3.9 \times 10^{-6}$ | $4.0 \times 10^{-6}$ | $8.6 \times$          |
| SiLU, GL match-DH <sup>†</sup>                      | $8.9 \times 10^{-6}$ | $4.9 \times 10^{-5}$ | $4.3 \times 10^{-5}$ | $3.6 \times 10^{-5}$ | $5.5 \times$          |
| <i>Other activations (DH default, for context):</i> |                      |                      |                      |                      |                       |
| Softplus( $\beta=1$ )                               | $1.3 \times 10^{-5}$ | $1.2 \times 10^{-4}$ | $1.8 \times 10^{-4}$ | $2.3 \times 10^{-4}$ | $8.8 \times$          |
| tanh                                                | $7.6 \times 10^{-5}$ | $1.2 \times 10^{-3}$ | $1.1 \times 10^{-3}$ | $9.8 \times 10^{-4}$ | $15.7 \times$         |
| ReLU                                                | $7.7 \times 10^{-4}$ | $1.1 \times 10^{-3}$ | $1.1 \times 10^{-3}$ | $1.1 \times 10^{-3}$ | $1.5 \times$          |
| $ \cdot $                                           | $1.8 \times 10^{-3}$ | $3.3 \times 10^{-3}$ | $2.4 \times 10^{-3}$ | $1.6 \times 10^{-3}$ | $1.9 \times$          |

<sup>†</sup> 2-epoch unconverged pilot; a converged 50-epoch GL run is queued and will replace this row in the camera-ready.

layers 2–3. What *does* differ between trained checkpoints is the *absolute floor*: training with a finer grid lowers  $L_0$  by  $\sim 160 \times$  relative to DH default and propagates that floor through subsequent layers, with the jump shape essentially preserved.

## 5.4 Mechanism: a $\delta/c$ toy model

Let  $\delta$  be the typical per- $S^2$ -Act perturbation of an  $\ell>0$  component, and let  $c \ll 1$  be the cross-degree coupling factor by which  $\ell>0$  corruption leaks into  $\ell=0$  (via RMSNorm’s all-degree normalisation and the SO(2) convolution’s  $m=0$  nn.Linear that mixes  $\ell=0$  with  $\ell>0$  coefficients).

- **Layer 0 input:**  $\ell=0$  error is exactly 0 (sphere\_embedding is rotation-invariant by construction);  $\ell>0$  error is exactly 0 (EdgeDegreeEmbedding is rotation-equivariant by construction).
- **Layer 0 output:** EquiformerV2’s SeparableS2Activation routes  $\ell=0$  through a scalar SiLU that does not see the  $S^2$  grid, so layer 0 has only *indirect*  $\ell=0$  contamination,  $L_0 \sim c \cdot \delta$ . The  $\ell>0$  component sees one  $S^2$ -Act forward, so it picks up the full  $\delta$ .
- **Layer 1 output:** The input now carries  $\ell>0$  corruption  $\sim \delta$ . Attention aggregates corrupted neighbour features through learned linear projections and produces new  $\ell=0$  values directly from the corrupted  $\ell>0$  input (a *first-order* path). So  $L_1 \sim O(1) \cdot \delta$ , and  $L_1/L_0 \sim 1/c$ , the inverse of the cross-degree coupling factor.

This explains both the empirical jump ratio and its activation dependence: smooth activations have small  $c$  (their  $\ell=0$  channels are mostly invariant, and RMSNorm fluctuations from  $\ell>0$  are second-order), so the jump is large (8–16 $\times$ ); non-smooth activations have large  $\delta$ , indirect leak is no longer perturbative, and the ratio collapses to 1.5–1.9 $\times$ . After layer 1 both  $\ell=0$  and  $\ell>0$  carry comparable errors; residual connections plus RMSNorm bound further growth and the relative error saturates.

## 5.5 Why infinite-grid-at-inference would not help

A natural objection: in the limit of an infinitely dense grid the  $S^2$ -Activation operator becomes exact, so the trained-model invariance error *should* approach zero. Why doesn't a denser inference grid visibly reduce the per-layer error in Table 2? Two facts together resolve this. (i) The kernel-level equivariance error at  $\ell_{\max}=6$ ,  $m_{\max}=2$  saturates at  $\sim 0.327$  regardless of grid; the floor comes from  $m_{\max}=2$  truncation, which discards  $|m| > 2$  components and is not a quadrature error any grid can fix. (ii) Trained features concentrate in the part of the basis  $m_{\max}=2$  *keeps* (low- $m$ ,  $\ell=0$ -dominated), so the truncation floor barely hits them. Their residual error is bounded above by  $\|c\| \cdot \|\ell>0 \text{ features}\|$ , and the second factor is small *because training drove it small* against the training-time grid. Inference-time grid swap perturbs the  $\ell>0$  features by  $\sim 1\%$ ; the corresponding  $\ell=0$  leak is the same percentage of an already-small number, hence the percent-level invariance of Table 2. Going to an infinitely dense grid at inference cannot drive the floor lower because there is no gradient telling the trained weights to make the  $\ell>0$  residual any smaller: that optimization happens only at training time.

## 5.6 Practical implications

The inference-time drop-in is therefore a *free wall-clock reduction at preserved task accuracy*, not a way to tighten trained-model equivariance. To extract the kernel-level equivariance gain in trained-model invariance, one trains with the better grid: GL match-DH at training time inherits the wall-clock benefit of §4 *and* drops the per-layer floor by 1–2 orders of magnitude. We have queued five 50-epoch SiLU+GL match-DH training runs; their results will replace the 2-epoch pilot row in Table 3 for the camera-ready.

## 6 Related Work

**The  $S^2$ -activation lineage.** SCN Zitnick et al. [2022] introduced the  $S^2$ -activation operator for SO(3)-equivariant atomistic networks; eSCN Passaro and Zitnick [2023] reduced its SO(3) convolutions to SO(2) for efficiency; EquiformerV2 Liao et al. [2024] integrated this into a transformer backbone and explicitly noted that “sampling-induced equivariance error remains an open challenge.” All three inherit e3nn’s equiangular DH grid without modification.

**Concurrent work, complementary levers.** *EquiformerV3* Liao et al. [2026] (April 2026, by the V2 authors) addresses the same equivariance problem at the *architectural* level: SwiGLU- $S^2$  replaces the  $S^2$  activation with a SwiGLU operating only on scalars and bilinear products, avoiding the grid round-trip entirely and achieving a 50.6% reduction in grid points while maintaining strict equivariance. It requires retraining and a new model. *EST* An et al. [2025] (May 2025) addresses it at the *sampling-pattern* level: replaces the e3nn grid with a Fibonacci lattice, observing that e3nn’s polar clustering wastes points. It also requires retraining. Our work targets a third, complementary lever—the *quadrature rule*—and operates as a drop-in change for both pretrained checkpoints and from-scratch training.

**Community awareness of the problem.** The fairchem issue tracker contains issue #1068 (March 2025) fairchem community [2025], where users reported the equivariance problem and discussed mitigations including “more grid points” and double precision, both prohibitively expensive. None of the proposed fixes considered alternative quadrature rules.

## 7 Limitations

**$m_{\max}$  truncation is a separate floor.** The kernel-level equivariance saturates at  $\sim 0.327$  at  $\ell_{\max}=6$ ,  $m_{\max}=2$  regardless of how dense the grid is. This is a truncation error from the eSCN reduction, not a quadrature error; no choice of quadrature can break it. Models that use  $m_{\max}=\ell_{\max}$  (no truncation) reach a much lower floor ( $\sim 4 \times 10^{-3}$  at  $\ell_{\max}=6$ ) where the GL advantage is even more pronounced.

**No fully-trained 12-layer GL run yet.** Our trained-checkpoint experiments use the smaller 4-layer expF architecture because we do not have a publicly available 12-layer fairchem-default QM9 checkpoint. The architecture-level wall-clock claim is weight-independent, so it transfers, but extending the per-layer floor data (Table 3) to 12 layers is left to future work.

**Lebedev does not fit the tensor-product einsum natively.** Lebedev  $d=21$  is the Pareto-optimal kernel-level choice in our benchmark (Appendix B), but its non-tensor-product structure requires reshape-as- $1 \times N$  within EquiformerV2’s einsum, which is suboptimal. Native support would require a small refactor of `S2Activation.forward`.

**Public-checkpoint loading hit module-shape mismatches.** We attempted to load the fairchem OC20 EquiformerV2-31M public checkpoint into our QM9 model wrapper; mismatches in the `rad_func.net` radial-function widths (`share_atom_edge_embedding`, etc.) prevented exact loading. The architecture-level wall-clock for that configuration is in Appendix A; the drop-in safety result remains established only on our own trained checkpoints.

## 8 Conclusion

We have argued that the choice of quadrature method in EquiformerV2’s  $S^2$  activation is a useful and underexplored lever: orthogonal to the architectural changes of EquiformerV3 and the sampling-pattern changes of EST. We instantiated two alternatives—Gauss–Legendre and Lebedev—in a drop-in `CustomS03Grid` module, and showed that switching to GL match-DH delivers  $DH \times 2$ ’s equivariance floor at 31.3% lower wall-clock per forward, with no retraining and no measurable change in trained-model predictions. Per-layer measurements expose a universal two-phase mechanism that explains both the safety of the inference-time swap and the complementary recommendation to train with a finer grid when one wants to lower trained-model invariance error—both consistent with our  $\delta/c$  toy model.

**Future work.** (i) A 50-epoch GL match-DH training sweep on the 12-layer fairchem default to close the trained-model invariance gap on the headline architecture. (ii) A native Lebedev path through EquiformerV2’s  $S^2$ -Act forward, eliminating the  $1 \times N$  reshape and allowing the kernel-Pareto-best Lebedev  $d=21$  to deploy without adaptation. (iii) A unified architectural+quadrature ablation combining EquiformerV3’s SwiGLU- $S^2$  with GL grids, since the two levers act on different parts of the activation pipeline and are expected to compound.

## References

- J. An et al. Equivariant spherical transformer for efficient molecular modeling. arXiv:2505.23086, 2025. May 2025.
- J. R. Driscoll and D. M. Healy. Computing Fourier transforms and convolutions on the 2-sphere. *Advances in Applied Mathematics*, 15(2):202–250, 1994.
- fairchem community. EquiformerV2Backbone has surprisingly large equivariance error. GitHub issue, FAIR-Chem/fairchem#1068, 2025. <https://github.com/FAIR-Chem/fairchem/issues/1068>.
- C. Hirt, M. Rexer, and S. Claessens. Ultra-high-degree surface spherical harmonic analysis using the Gauss–Legendre and the Driscoll/Healy quadrature theorem and application to planetary topography models of Earth, Mars and Moon. *Surveys in Geophysics*, 36(6):803–830, 2015.
- Y.-L. Liao, B. Wood, A. Das, and T. Smidt. EquiformerV2: Improved equivariant transformer for scaling to higher-degree representations. In *International Conference on Learning Representations*, 2024.
- Y.-L. Liao, A. J. Hoffman, S. C. Shen, A. Duval, S. W. Norwood, and T. Smidt. EquiformerV3: Scaling efficient, expressive, and general  $SE(3)$ -equivariant graph attention transformers. arXiv:2604.09130, 2026. April 2026.
- S. Passaro and C. L. Zitnick. Reducing  $SO(3)$  convolutions to  $SO(2)$  for efficient equivariant GNNs. In *International Conference on Machine Learning*, pages 27420–27438. PMLR, 2023.
- M. A. Wieczorek and M. Meschede. SHTools: Tools for working with spherical harmonics. *Geochemistry, Geophysics, Geosystems*, 19(8):2574–2592, 2018.



## A Scaling savings across architectures

The 31% saving in the headline is for the 12-layer fairchem-default architecture. Two other published configurations show how the saving scales with depth and band limit (Figure 2): **QM9-small** (4 layers / 64 channels /  $\ell_{\max}=4$ , our trained-checkpoint testbed) shows +15.4%; **OC20 EquiformerV2-31M** (8 layers / 128 channels /  $\ell_{\max}=4$ , the public OC20 checkpoint we attempted to load) shows +13.9%; the **fairchem-default 12-layer** model shows +31.2%. The pattern is monotonic in  $\ell_{\max}$ : at  $\ell_{\max}=4$  the DH  $2\times$  grid is  $20\times 20 = 400$  points (a  $5.7\text{--}8\times$  increase over default), while at  $\ell_{\max}=6$  DH  $2\times$  is  $28\times 28 = 784$  points (an  $11.2\times$  increase). Larger and deeper models benefit more from the swap because the  $S^2$ -activation kernel becomes a larger fraction of forward time.

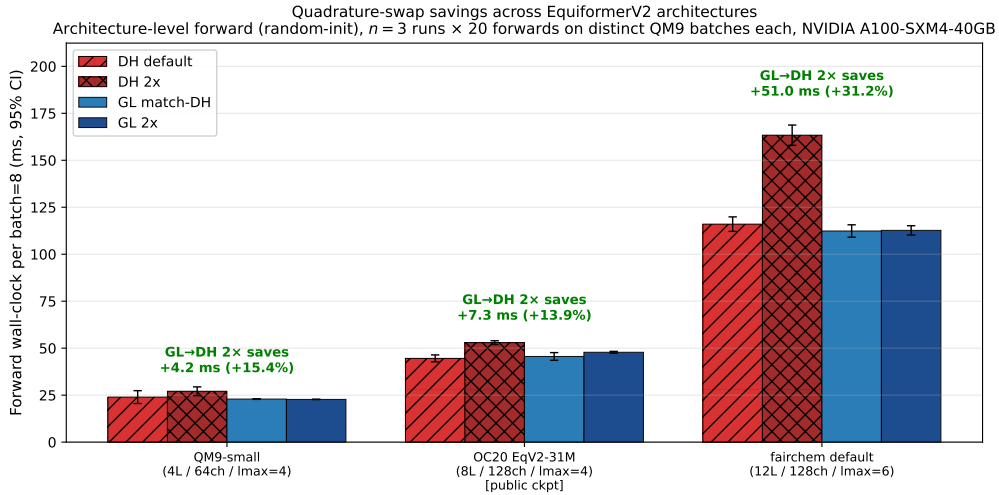


Figure 2: Forward wall-clock per QM9 batch 8 across three published EquiformerV2 architectures, for four grid configurations.  $n = 3 \text{ runs} \times 20 \text{ measured forwards}$  on distinct batches per run, error bars 95% CI on run means. Architecture-level (random-init) timing.

## B Why FFT does not save DH at this scale

DH’s classical algorithmic justification is the FFT-based fast SH transform. We give three observations explaining why this advantage is not realised in EquiformerV2.

(i) **e3nn implements only the longitude FFT.** e3nn’s `ToS2Grid.forward` (e3nn/o3/\_s2grid.py:367) takes the FFT path along longitude only when  $N_\alpha$  is odd and  $\geq 2\ell_{\max} + 1$ ; the Legendre direction is always a dense matmul. The full Driscoll–Healy fast Legendre transform is not implemented in e3nn (and in practice has a high break-even  $\ell_{\max} \gtrsim 64$  on GPU).

(ii) **EquiformerV2 bypasses the FFT entirely.** `S03_Grid` (lines 529–578 of fairchem’s `so3.py`) calls `ToS2Grid` only at construction time, collapses `shb` and `sha` into a single dense `to_grid_mat` of shape  $[N_\beta, N_\alpha, n_{\text{coeffs}}]$ , and applies it at runtime via a single `einsum`. The FFT branch in `ToS2Grid.forward` is never reached.

(iii) **Wiring up the FFT path makes the kernel slower.** We re-implemented `S2Activation` so that it explicitly invokes `ToS2Grid.forward` / `FromS2Grid.forward` at runtime and benchmarked against the dense path at the same grid size. At a  $14 \times 13 = 182$ -point grid, the FFT path is  $7\times$

slower than dense matmul (Table 4). The  $m_{\max}$ -cropping inflate/scatter step plus a much larger shb einsum (over the full  $(\ell_{\max} + 1)^2 = 49$  basis instead of the cropped 29 used by the dense path) wipes out the savings of the alpha FFT at  $N_\alpha = 13$ .

The same table shows that Lebedev at degree 21 (170 points, non-tensor-product) is the kernel-level Pareto winner: same equivariance floor as DH  $2\times$  and GL match-DH, at 28% lower kernel time than GL match-DH and 33% lower than DH-dense match.

Table 4: Kernel-level  $S^2$ -Activation benchmark: equivariance error and per-call time for four quadrature methods at  $\ell_{\max}=6$ ,  $m_{\max}=2$ , SiLU activation, batch 1024, 128 channels, NVIDIA A100-SXM4-40GB. Equivariance error is averaged over 5 random rotations  $\times$  10 random inputs; kernel time is mean  $\pm$  95% CI over 3 runs  $\times$  20 forwards each.

| Method                              | Pts | Equiv err                               | Kernel (ms)                       |
|-------------------------------------|-----|-----------------------------------------|-----------------------------------|
| DH-dense default ( $N_\alpha = 5$ ) | 70  | $4.24 \times 10^{-1}$                   | $0.34 \pm 0.00$                   |
| DH-dense match ( $N_\alpha = 13$ )  | 182 | $3.35 \times 10^{-1}$                   | $0.62 \pm 0.01$                   |
| <b>DH-FFT</b> ( $N_\alpha = 13$ )   | 182 | $3.35 \times 10^{-1}$                   | <b><math>4.32 \pm 0.05</math></b> |
| GL match-DH ( $N_\alpha = 13$ )     | 182 | $3.34 \times 10^{-1}$                   | $0.58 \pm 0.00$                   |
| Lebedev ( $d = 13$ )                | 74  | $5.11 \times 10^{-1}$                   | $0.25 \pm 0.00$                   |
| <b>Lebedev</b> ( $d = 21$ )         | 170 | <b><math>3.35 \times 10^{-1}</math></b> | <b><math>0.41 \pm 0.00</math></b> |

## C Reproducibility

All experiments run on Perlmutter under the module load `pytorch/2.6.0-1` environment (Python 3.12.12, PyTorch 2.6.0, CUDA 12.4, e3nn 0.4.4, fairchem-core 1.10.0, torch\_geometric 2.7.0). Two non-obvious gotchas: PyTorch 2.6’s `weights_only=True` default breaks e3nn’s `constants.pt`, requiring `torch.serialization.add_safe_globals([slice])` before any e3nn import; and the 12-layer / `lmax=6` model fragments the CUDA allocator unless `PYTORCH_CUDA_ALLOC_CONF=expandable_segments=True`.

The exact commands to reproduce the four benchmarks of this paper are:

```
# Headline Pareto (Section 4)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments=True \
python3 experiments/expG_verify_speedup_v2.py \
--batch_size 8 --n_runs 5 --n_iter 30 --n_warmup 10
# Drop-in safety on trained checkpoints (Section 5.1-5.2)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments=True \
python3 experiments/expG_dropin_pretrained.py \
--seeds 123 456 789 1024 --activation SiLU --orig_grid default
# Architecture sweep (Appendix A)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments=True \
python3 experiments/expG_sweep_models.py
# Four-way kernel benchmark (Appendix B)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments=True \
python3 experiments/expG_four_way.py
```